Axona Services Guide

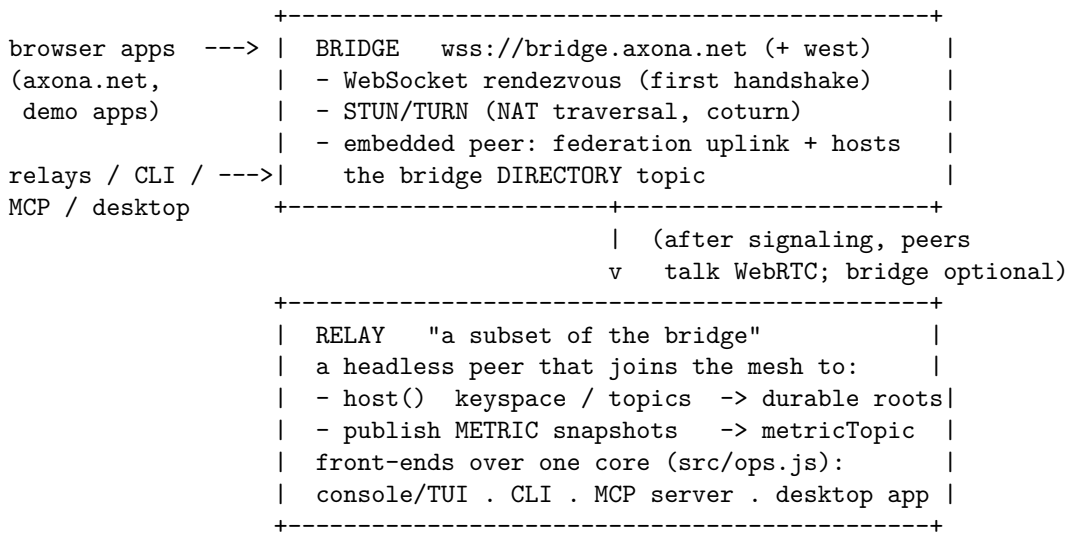
The other programmer-guide documents teach the **library** — how to build your own peer with `@axona/protocol` and speak pub/sub. This guide covers the **services**: the ready-made programs you *run* (or rent) rather than write — the signaling **bridge**, the **relay** and its four front-ends (console, CLI, MCP server, desktop app), the **directory/federation** that ties bridges together, and the **PoW collector**. If you are operating an Axona deployment, wiring an agent into the network, or just want a topic to stay alive when no browser tab is open, this is the document for you.

- **Protocol kernel**: @axona/protocol (v3.6.0)
- **Wire version**: 3.0 (`WIRE_VERSION`); kernel version 3.6.0 (`KERNEL_VERSION`)
- **Live network**: `wss://bridge.axona.net` (east) + `wss://bridge-west.axona.net` (west) — a federated pair
- **Companion docs**:
  - Quick Start · Programmer Guide · API Reference — the library.
  - Architecture — how the bridge + transport work under the hood.

Library vs. services. A *peer* is the unit the library gives you: an identity + a transport + a pub/sub manager, running inside a browser tab, a Node process, or an agent. The library is enough to build a complete app. The **services** exist because some jobs outlive a single tab: brokering the first WebRTC handshake between two strangers (the **bridge**), keeping a topic's messages available when every author has closed their laptop (a **relay** that `host()`s the topic), and letting non-browser callers — shells, agents, GUIs — publish and subscribe (the **CLI / MCP / desktop** front-ends). None of them is privileged: every service is just a peer that has volunteered to stay online and take on a role.

---

1. The service map



Service	Package / entry	What it is	Runs where
Bridge	`axona-bridge` (`src/server.js`)	Signaling rendezvous + STUN/TURN + a federating, directory-hosting peer	A server (Docker stack)

Service	Package / entry	What it is	Runs where
Relay — console/TUI	`axona-relay` (`src/index.js`, bin `axona-relay`)	Headless node that hosts topics + publishes metrics; live dashboard	A box you keep online
Relay — CLI	`axona-cli` (`src/cli.js`)	One-shot `pub` / `sub` / `pull` from a shell or script	Anywhere with Node
Relay — MCP server	`axona-mcp` (`src/mcp.js`)	The same ops as native **agent tools** over MCP/stdio	Spawned by the agent host
Relay — desktop app	`axona-relay/desktop` (Electron)	A GUI wrapper around the relay	A laptop/desktop
PoW collector	`axona-relay/pow-collector` (`npm run collect`)	Subscribes to the PoW-bench results topic and archives them	A box you keep online
Directory / federation	kernel `bridgeDirectory.js`	A *behavior*, not a process: bridges advertise, clients fail over	Inside bridges + clients

The same `@axona/protocol` kernel runs in every box above. There is no special "server build" — a bridge is a peer that also terminates WebSockets and TURN; a relay is a peer that also `host()`s and publishes metrics.

---

2. The Bridge

What it is

A **bridge** is the only piece of fixed infrastructure Axona needs, and even it is needed only at the **start** of a connection. Two browsers that have never met cannot exchange WebRTC offer/answer SDP on their own — they need a mutually reachable rendezvous. The bridge is that rendezvous. It does three jobs:

1. **WebSocket signaling** — relays the offer/answer/ICE handshake between two peers until their direct WebRTC data channel is open. After that the bridge is **out of the data path**; peers talk peer-to-peer over the mesh.
2. **STUN/TURN** — a bundled **coturn** gives peers their public reflexive address (STUN) and, for the ~10–15% of NAT pairs that can't connect directly, relays the media (TURN). The bridge mints short-lived TURN credentials in its welcome frame.
3. **An embedded peer** — the bridge process also *joins its own network* as a node. That embedded peer **federates** (an outbound uplink to a sibling bridge, §5) and **hosts the bridge directory topic** so clients can discover other bridges and fail over.

   A bridge is a convenience, not an authority. It cannot read message payloads (they're end-to-end between mesh peers), cannot forge a publish (every envelope is author-signed), and any number of independent bridges can coexist — clients rank and fail over between them.

Running one

Production bridges run as a **Docker Compose stack** — `bridge` + **Caddy** (automatic TLS) + **coturn** (TURN) — from the `axona-bridge` repo. The one-command path for a fresh Ubuntu/Debian host:

```
curl -fsSL https://raw.githubusercontent.com/axona-net/axona-bridge/main/deploy/install.sh \
  | sudo BRIDGE_DOMAIN=bridge.example.net LETSENCRYPT_EMAIL=you@example.net bash
```

or, with the repo checked out:

```
cd /opt/axona-bridge
docker compose build        # builds first; the old container keeps serving
docker compose up -d        # fast swap; Caddy keeps its certificate
curl https://bridge.example.net/healthz     # → {"status":"ok","version":…,"kernelVersion":"3.6.0"}
```

Full provisioning options (the installer, the Docker bundle, and a manual walkthrough) are in `axona-bridge/deploy/INSTALL.md`.

Configuration

Bridge config lives in `/etc/axona-bridge.env` (systemd) or the compose `.env`:

Var	Meaning
`DOMAIN` / `PUBLIC_IP`	Public hostname + IP (TLS + TURN advertisement)
`TURN_AUTH_SECRET`	Shared secret with coturn's `static-auth-secret`; the bridge mints creds from it
`BRIDGE_PUBLIC_URL`	The `wss://` URL this bridge advertises in the directory (e.g. `wss://bridge-west.axona.net`)
`BRIDGE_DIRECTORY`	`on` (advertise + host the directory) or `off` (isolated fleet — testnet uses `off`)
`BRIDGE_LAT` / `BRIDGE_LNG` / `REGION_LABEL`	The bridge's geo anchor + a human label for its directory entry
`MIN_PEER_VERSION`	Lowest wire version admitted; older peers get `UPGRADE_REQUIRED` (WS close 4426)
`HOST`	Bind address — production sets `127.0.0.1` so the reverse proxy is the only ingress
`HEALTHZ_TOKEN`	Gates the **full** `/healthz` body + `/diag`; unauthenticated `/healthz` returns only `{status,version,kernelVersion}`

Health & introspection

- `GET /healthz` — unauthenticated, returns `{status, version, kernelVersion}`.
- `GET /healthz` with header `X-Healthz-Token: <token>` — the full body: synaptome size, directory `{enabled,url}`, uplink status, peer counts.
- `GET /diag` (token-gated) — deeper diagnostics.

---

3. The Relay

What it is

A **relay is a subset of the bridge**: the embedded-peer half, without the WebSocket/TURN server. It is a headless node you keep online so the network has durable, well-placed members. Two jobs:

1. **Hosting** — `host()` makes the relay a **root** for a keyspace region (or for named topics) *without subscribing*, so it stores-and-serves a topic's messages even when it doesn't consume them. This is what keeps a topic's backlog answerable after every author closes their tab. By default a relay hosts its region's keyspace (`0x89` for `useast`).
2. **Metrics** — every ~5 minutes the relay walks its rooted **open** topics and publishes a signed snapshot (`{current_count, subscribers, bytes}`) to the derived `metricTopic(T)`, so clients can `sub()` for

live counts + a rolling ~48 h trend instead of polling. (Owned topics are skipped — their counts are owner-private.)

Running one

```
cd axona-relay
npm install
npm start          # live dashboard (interactive TTY)
npm run probe      # plain timestamped log lines (RELAY_TUI=0) - for piping / services
```

Run it again in another terminal for a second node — the **first** instance claims the persistent identity (stable nodeId, `identity.<region>.json`); each **additional** instance mints a fresh ephemeral identity in the same region. So you get one well-known node plus as many throwaway nodes as you want.

Configuration (env)

Var	Default	Meaning
`RELAY_NETWORK`	`prod`	Bootstrap network: `prod` (`bridge.axona.net`) or `testnet`
`BRIDGE_URL`	—	Explicit bridge URL; **overrides** `RELAY_NETWORK`
`RELAY_REGION`	—	`auto` (detect), a region name (`useast`), or code (`0x89`) — sets the nodeId geo prefix
`RELAY_LAT` / `RELAY_LNG`	`37.77/-122.42`	Geo prefix by coordinate (if `RELAY_REGION` unset). Default = SF (`uswest`)
`RELAY_IDENTITY_PATH`	`./identity.<region>.json`	Persisted keypair (stable nodeId)
`RELAY_HOST_KEYSPACE`	`1`	Host the region's whole keyspace (set `0` to host only `RELAY_TOPICS`)
`RELAY_TOPICS`	—	Comma-separated topic names to host explicitly
`RELAY_METRICS`	`1`	Publish metric snapshots (`0` to disable)
`RELAY_METRICS_INTERVAL_MS`	~300000	Metric publish cadence
`RELAY_TUI`	auto	`1` force dashboard, `0` force plain log

Bridge precedence: `BRIDGE_URL` › `RELAY_NETWORK` › `prod`. Region precedence: `RELAY_REGION` › `RELAY_LAT`/`RELAY_LNG` › default SF.

Run as a service (systemd)

```
# /etc/systemd/system/axona-relay.service
[Service]
Environment=RELAY_REGION=useast RELAY_TUI=0
ExecStart=/usr/bin/node /opt/axona-relay/src/index.js
Restart=always
```

Operator note — process count. Each relay forks **one worker child**, so a healthy N-node fleet is **2 × N** `node src/index.js` processes (a parent + worker per node). Judge fleet health by the distinct **bridge connections** in the logs, not the raw process count. If the **desktop app** (below) is also running, it carries *its own* embedded relay that respawns if killed — expect one extra.

4. Relay front-ends

All three non-dashboard front-ends share one core module (`src/ops.js` — `publish` / `subscribe` / `pull`), so they behave identically; they differ only in how you call them.

4a. CLI (`axona-cli`)

One-shot pub/sub/pull from a shell or script — no long-lived process:

```
npm run pub  -- --topic us-east/hello --message "hi"          # prints { ok, msgId, … }
npm run sub  -- --topic us-east/hello --seconds 20            # JSON lines for N seconds
npm run pull -- --topic us-east/hello                         # the single latest message
```

Good for cron jobs, CI checks, shell glue, and quick manual probes.

4b. MCP server (`axona-mcp`)

`src/mcp.js` is a **Model Context Protocol** server that exposes Axona pub/sub as **native agent tools**, so Claude Code (and any MCP-speaking agent) gets first-class `axona_*` tools instead of shelling out. It speaks JSON-RPC over **stdio** (stdout = protocol, stderr = logs). Each call connects a *fresh ephemeral peer* to the live network, does one job, tears down, and returns JSON — it holds no persistent peer.

Tool	Args	Returns
`axona_publish`	`topic`, `message`, `region?`	`{ ok, msgId }`
`axona_subscribe`	`topic`, `region?`, `seconds?` (1–120), `since?` (`all`|`new`)	`{ received, messages[] }`
`axona_pull`	`topic`, `region?`	`{ found, message, msgId }`

Region defaults to `useast` (`0x89`); subscribers must use the **same region** as the publisher. It targets **production** by default and interoperates with the live apps — publishing to `us-east/hello-world` shows up in the demo.axona.net feed.

Register it as a project-scoped MCP server with a `.mcp.json` at your repo root:

```
{
  "mcpServers": {
    "axona": { "command": "node", "args": ["/abs/path/to/axona-relay/src/mcp.js"] }
  }
}
```

Point it at testnet (or a specific bridge) with an `env` block:

```
{
  "mcpServers": {
    "axona": {
      "command": "node",
      "args": ["/abs/path/to/axona-relay/src/mcp.js"],
      "env": { "RELAY_NETWORK": "testnet" }
    }
  }
}
```

4c. Desktop app (`axona-relay/desktop`)

An **Electron** GUI wrapper around the relay — the same hosting + metrics node, with a window instead of a terminal. Useful for non-operators who want to contribute a durable node by leaving an app open. Because it embeds a relay, a running desktop app adds one node to your local fleet (and respawns it on crash).

---

5. Directory & federation

A single bridge is a single point of failure. Axona avoids that with two cooperating behaviors, both built into the bridge's embedded peer:

- **Directory.** Each bridge with `BRIDGE_DIRECTORY=on` publishes a signed entry (`{url, lat, lng, label, ver, ts}`) to the well-known open topic `axona:bridge-directory` (pinned to the `useast` region so everyone derives the same Topic ID) and `host()`s that topic so the entry survives. At launch a client collects the directory, ranks candidates (configured roots → known-good by reputation → fresh from the directory), and **fails over** to a saved alternate if its primary is unreachable.
- **Federation.** A bridge with the directory on also **uplinks** — an outbound `webTransport` connection *into* a sibling bridge — so the two bridges share one connectome. A client connected to **either** bridge can then discover and reach peers on **both**. In the live network, `bridge-west` uplinks to `bridge.axona.net`; the east bridge stays uplink-less as the seed.

  Verify federation against a **warm** topic (both bridges host + republish `axona:bridge-directory`), never a cold fresh topic — a cold topic has no roots yet and reads as a false zero.

---

6. The PoW collector

`pow-collector.js` (`npm run collect`) is a small standing service that subscribes to the proof-of-work benchmark **results** topic and archives each published result for later analysis. It's optional infrastructure for the PoW research line, not part of the core pub/sub path. (It can wedge silently — alive but not receiving — so a restart with `since:'all'` backfills the retained backlog.)

---

7. Choosing what to run

You want to…	Run
Let strangers connect to your app's network	A **bridge** (or use the live `bridge.axona.net`)
Keep a topic's messages alive when no author is online	A **relay** that `host()`s it
Publish live subscriber counts for a topic	A **relay** (metrics on — the default)
Publish/subscribe from a shell, cron, or CI	The **CLI** (`axona-cli`)
Give an AI agent native Axona tools	The **MCP server** (`axona-mcp`)
Contribute a node from a desktop, no terminal	The **desktop app**
Survive a bridge outage	Two **federated bridges** + the directory

Most application developers run **none** of these in development — they point the library at the live `bridge.axona.net` and let the existing relay fleet host their topics. You reach for self-hosted services when you want an isolated network, guaranteed topic durability, or an agent/automation integration.

---

8. Are there analogous systems?

If you know other realtime/decentralized stacks, the Axona services map onto patterns you've already seen:

Axona service	Closest analogue elsewhere
Bridge (signaling + TURN)	WebRTC **STUN/TURN** servers; **libp2p** circuit-relay v2 / rendezvous
Relay (durable topic host)	An **MQTT broker** / **NATS** server with retained messages — but decentralized and unprivileged
Bridge directory	**DNS seeds / bootstrap node lists** (Bitcoin, IPFS)
MCP server	Any other **MCP integration** (GitHub, Slack) — a thin agent-facing adapter over the library

The difference in every row is the same: in those systems the server is an *authority* (it owns the namespace, the messages, or the membership). In Axona the service is a *volunteer* — a peer that took on a role and can be replaced by any other peer that takes on the same role.

Where to go next

- **Programmer Guide** — build the peer that talks to these services.
- **API Reference** — `host()` / `unhost()`, `metricTopic()`, and the rest of the public surface.
- `axona-bridge/deploy/INSTALL.md` — provision a bridge (installer, Docker, or manual).
- `axona-relay/README.md` — the relay's full configuration + dashboard reference.

Found a gap in this guide? Open an issue at https://github.com/axona-net/axona-docs.